Date :

```
void run() { System.out.println("Vehicle is running"); }
}
// creating a child class
class Bike2 extends Vehicle {
// defining the same method as in the parent class
 void run() { System.out.println("Bike is running safely"); }
 public static void main (String args[]) {
  Bike2 obj = new Bike2(); // creating object
  obj.run(); // calling method
 }
}
```

Output:

Bike is running safely.

7.3 Final keyword in Java :

The Final keyword in Java is used to restrict the user. The java final keyword can be used in many context. Final can be :

1. Variable
2. Method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these.

7.4 Runtime polymorphism :

Polymorphism in Java is a concept by which

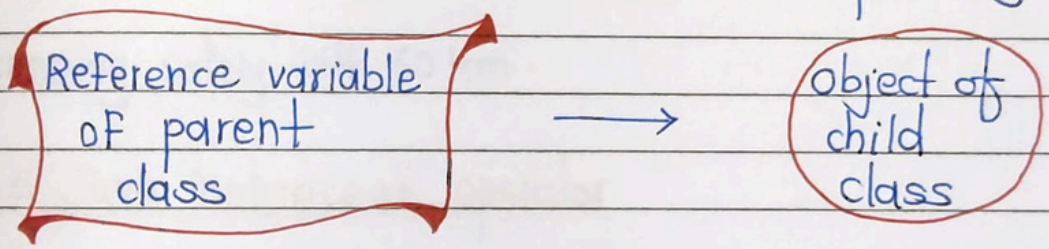
Date : _____

we can perform a single action in different ways. Polymorphism is derived from 2 Greek words: Poly and morphs. The word "poly" means many and "morph" means forms.

Runtime polymorphism or Dynamic Method dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile time.
In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being reffered to by the reference variable.

Upcasting:

If the reference variable of parent class refers to the object of child class, it is known as upcasting.




```
class A{}
class B extends A{}

A a=new B(); //upcasting
```

For upcasting, we can use the reference varible of class type or an interface type. For example:

```
interface I{}
```

Date : _____

```
class A{}
class B extends A implements I{}
```

Java Runtime Polymorphism Example:

```
class Bike{
  void Run(){system.out.println("running");}
}
class splendor extends Bike{
  void run(){system.out.println("running safely with 60km");}
public static void main(String args[]){
  Bike b = new Splendor();
  b.run();
 }
}
```

Output:

running safely with 60 km.

7.5 Java instanceof operator

The java instanceof operators is used to test whether the object is an instance of the specified type(class or subclass or interface)

The instanceof in Java is also known as type comparison operator because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has null value, it returns false

Date :

example:

```
class Simple1{
public static void main (string args[]){
simple1 s =new Simple1();
system.out.println (s instance of simple1);
}
}
```

output:

True.